

患者详情页面设计文档

1. 1. Story功能描述

1.1. 系统名称

生命体征监测系统 - 患者详情模块

1.2. 类型

医疗健康监测应用的核心功能模块

1.3. 在项目中的地位

作为生命体征监测系统的关键组件，负责展示单个患者的详细信息、实时生命体征数据和历史数据趋势。

1.4. 任务目标

- 展示患者基本信息
- 显示设备绑定状态和设备详情
- 实时监测和展示生命体征数据
- 提供生命体征数据的历史记录查看
- 支持阈值设置和异常提醒

2. 2. 详细设计

2.1. 2.1 界面设计

02:58

100

< 患者详情

正在就诊



qi

女 | 24岁

住院号: 2504160001 [查看阈值](#)

设备信息

设备名称: Device01

状态: 使用中

设备ID: A001

电量: 100%

实时生命体征

[点击查看趋势](#)

体温

异常

38.1 °C

血压

异常

72/45 mmHg

心率

异常

49 次/分

血氧

正常

96 %



查看所有体征数据

2.1.1. 布局结构

2.1.1.1. 顶部导航栏

```
1 Row() {  
2   Image($r('app.media.ic_public_arrow_left'))  
3   Text('患者详情')  
4   // 就诊状态标识  
5 }
```

2.1.1.2. 患者信息卡片

```
1 Column() {  
2   Row() {  
3     Image($r('app.media.ic_public_avatar'))  
4     Column() {  
5       Text(name)  
6       Text(`${gender} | ${age}岁`)  
7       Text(`住院号: ${hospitalId}`)  
8     }  
9   }  
10 }
```

2.1.1.3. 设备信息卡片

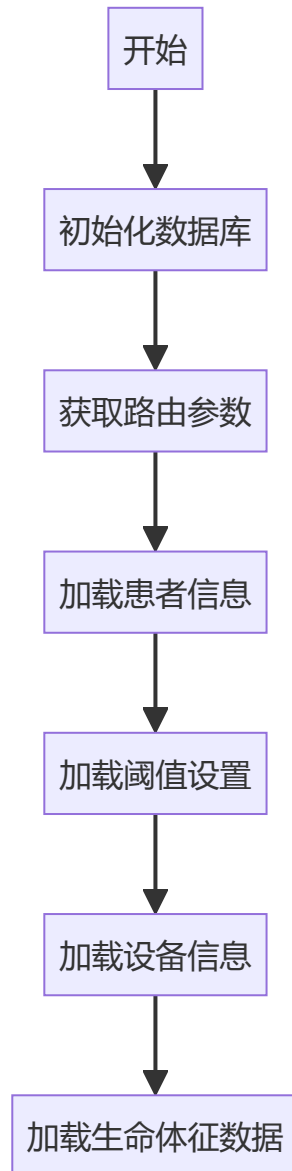
```
1 Column() {  
2   Row() {  
3     Text('设备信息')  
4     // 设备状态和详情  
5   }  
6 }
```

2.1.1.4. 生命体征数据卡片

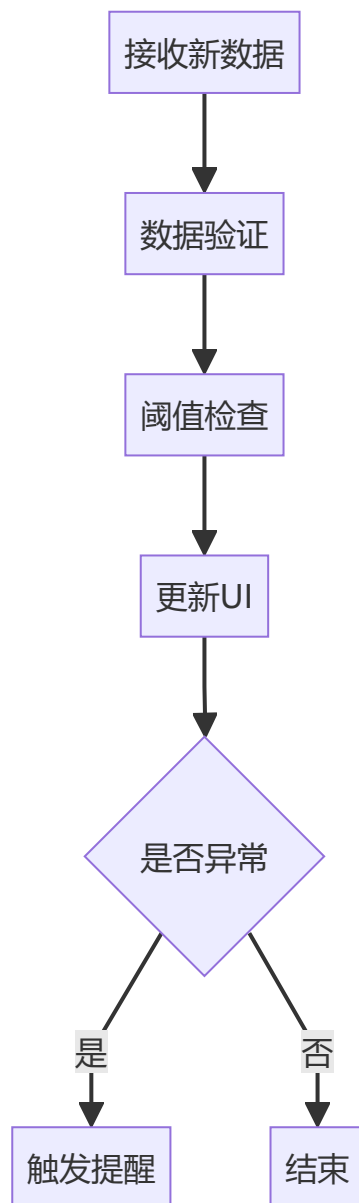
```
1 Column() {  
2   Grid() {  
3     GridItem('体温')  
4     GridItem('血压')  
5     GridItem('心率')  
6     GridItem('血氧')  
7   }  
8 }
```

2.2. 2.2 内部处理流程

2.2.1. 页面初始化流程



2.2.2. 数据更新流程



2.3. 2.3 数据流

2.3.1. 输入数据结构

```
1 interface VisitRecord {
2     hospitalId: string;
3     name: string;
4     idCard: string;
5     deviceId?: string;
6     diagnosis?: string;
7 }
8
9 interface VitalSigns {
10     temperature: number;
11     bloodPressure: {
```

```

12     systolic: number;
13     diastolic: number;
14 };
15     heartRate: number;
16     bloodOxygen: number;
17 }

```

2.3.2. 输出数据结构

```

1 interface TrendData {
2     type: string;
3     title: string;
4     hospitalId: string;
5     data: Array<{
6         time: string;
7         value: number;
8     }>;
9 }

```

2.4. 2.4 算法描述

2.4.1. 异常检测算法

```

1 function checkAbnormal(value: number, threshold: ThresholdData): boolean {
2     return value < threshold.min || value > threshold.max;
3 }

```

2.4.2. 数据格式化算法

```

1 function formatVitalSigns(data: VitalSigns): CombinedVitalSignData {
2     return {
3         measureTime: formatTime(data.timestamp),
4         temperature: formatTemperature(data.temperature),
5         bloodPressure: formatBloodPressure(data.bloodPressure),
6         heartRate: formatHeartRate(data.heartRate),
7         bloodOxygen: formatBloodOxygen(data.bloodOxygen),
8         hasAbnormal: checkAnyAbnormal(data)
9     };
10 }

```

2.5. 2.5 外部接口

2.5.1. 数据库接口

```
1 // 生命体征数据接口
2 interface VitalSignsDB {
3     getTemperaturesByHospitalId(hospitalId: string): Promise<TemperatureData[]>;
4     getBloodPressuresByHospitalId(hospitalId: string): Promise<BloodPressureData[]>;
5     getHeartRatesByHospitalId(hospitalId: string): Promise<HeartRateData[]>;
6     getBloodOxygensByHospitalId(hospitalId: string): Promise<BloodOxygenData[]>;
7 }
8
9 // 阈值数据接口
10 interface ThresholdDB {
11     getThresholdByHospitalId(hospitalId: string): Promise<ThresholdData>;
12     insertThreshold(threshold: ThresholdData): Promise<boolean>;
13 }
```

2.5.2. 路由接口

```
1 interface RouterInterface {
2     pushUrl(params: {
3         url: string;
4         params: {
5             type: string;
6             title: string;
7             hospitalId: string;
8         };
9     }): void;
10 }
```

2.6. 2.6 其他设计考虑

2.6.1. 性能优化

1. 数据缓存策略

- 使用 `@State` 装饰器管理状态
- 按需加载历史数据
- 分页加载大量数据

2. UI渲染优化

- 使用 `LazyForEach` 懒加载列表项
- 合理使用生命周期钩子

2.6.2. 安全性设计

1. 数据安全

- 敏感信息加密存储
- 数据访问权限控制
- 输入数据验证

2. 异常处理

```
1  try {  
2      await this.loadData();  
3  } catch (error) {  
4      console.error('数据加载失败:', error);  
5      promptAction.showToast({ message: '加载失败' });  
6  }
```

2.6.3. 可维护性

1. 代码组织

- 模块化设计
- 组件复用
- 清晰的注释

2. 扩展性

- 支持新增生命体征类型
- 可配置的阈值规则
- 灵活的UI布局

3.

3. 版本历史

版本号	日期	描述	作者
v1.0.0	2024-01-01	初始版本	Team
v1.1.0	2024-01-15	添加实时监测功能	Team